# 5 Data tidying

## 5.1 Introduction

"Happy families are all alike; every unhappy family is unhappy in its own way." — *Leo Tolstoy*

"Tidy datasets are all alike, but every messy dataset is messy in its own way." — *Hadley Wickham*

```
library(tidyverse)
```

## 5.2 Tidy data

There are three interrelated rules that make a dataset tidy:

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.

This is Codd's 3rd normal form, but with the constraints framed in statistical language, and the focus put on a single dataset rather than the many connected datasets common in relational databases. Messy data is any other arrangement of the data.

https://tidyr.tidyverse.org/

For a given dataset, it's usually easy to figure out what are observations and what are variables, but it is surprisingly difficult to precisely define variables and observations in general. For example, if the columns in the classroom data were height and weight we would have been happy to call them variables. If the columns were height and width, it would be less clear cut, as we might think of height and width as values of a dimension variable. If the columns were home phone and work phone, we could treat these as two variables, but in a fraud detection environment we might want variables phone number and number type because the use of one phone number for multiple people might suggest fraud. A general rule of thumb is that it is easier to describe functional relationships between variables (e.g., z is a linear combination of x and y, density is the ratio of weight to volume) than between rows, and it is easier to make

comparisons between groups of observations (e.g., average of group a vs. average of group b) than between groups of columns.

Packages in the tidyverse are designed to work with tidy data:

```
# Compute rate per 10,000
table1 |>
  mutate(rate = cases / population * 10000)
```

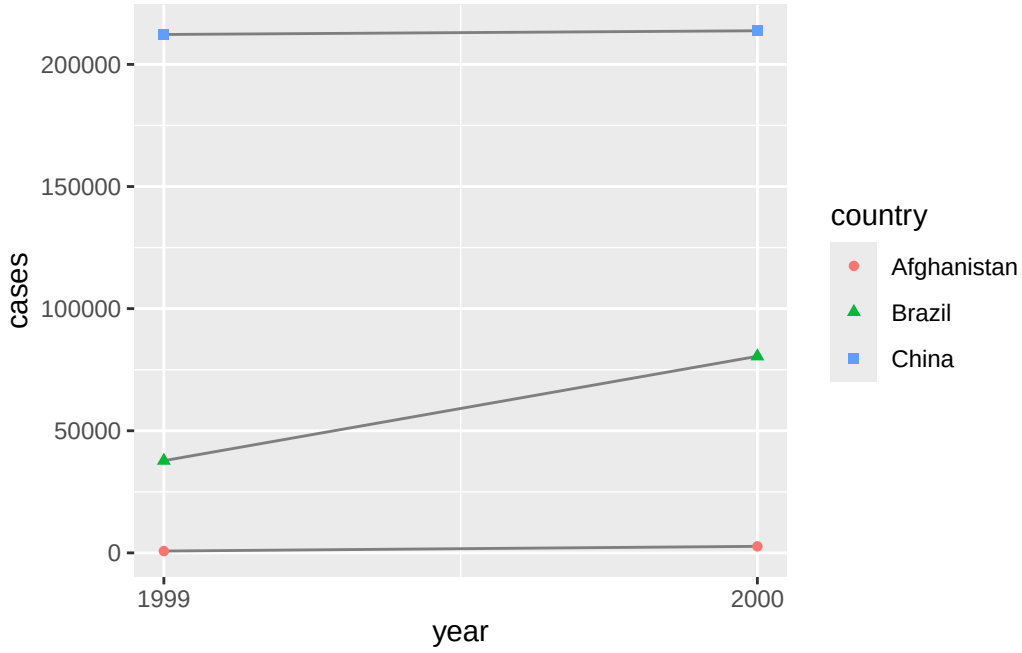
```
# A tibble: 6 x 5
  country      year  cases population  rate
  <chr>       <dbl>  <dbl>      <dbl> <dbl>
1 Afghanistan  1999    745   19987071 0.373
2 Afghanistan  2000   2666   20595360 1.29
3 Brazil       1999  37737  172006362 2.19
4 Brazil       2000  80488  174504898 4.61
5 China        1999 212258 1272915272 1.67
6 China        2000 213766 1280428583 1.67
```

```
# Compute total cases per year
table1 |>
  group_by(year) |>
  summarize(total_cases = sum(cases))
```

```
# A tibble: 2 x 2
   year total_cases
  <dbl>       <dbl>
1  1999      250740
2  2000      296920
```

```
# Visualize changes over time
ggplot(
  table1,
  aes(x = year, y = cases)
) +
  geom_line(
    aes(group = country),
    color = "grey50"
  ) +
  geom_point(
    aes(color = country, shape = country)
```

```
  ) +
  # x-axis breaks at 1999 and 2000
  scale_x_continuous(breaks = c(1999, 2000))
```




## Exercises

**1. For each of the sample tables, describe what each observation and each column represents.**

In table1, each row is one country–year observation. It tells, for a given country and year, how many cases of a disease were recorded and what the total population was.

Columns:

- country: Country name (e.g. Afghanistan, Brazil).
- year: Calendar year of data collection.
- cases: Number of people with the disease in that country and year.
- population: Total number of people living in that country in that year.

Conceptually, table1 is already tidy: one row per country–year, one column per variable.

```
table1
```

```
# A tibble: 6 x 4
  country      year  cases population
  <chr>       <dbl>  <dbl>      <dbl>
1 Afghanistan  1999    745   19987071
2 Afghanistan  2000   2666   20595360
3 Brazil       1999  37737  172006362
4 Brazil       2000  80488  174504898
5 China        1999 212258 1272915272
6 China        2000 213766 1280428583
```

In table2, each row is still one country–year–type observation, but the disease and population counts are stacked into a longer format.

Columns:

- country: Country name, same meaning as in table1.
- year: Calendar year, same meaning as in table1.
- type: Indicates which kind of count the row refers to, either "cases" or "population".
- count: The numeric value corresponding to type (number of disease cases if type == "cases", or total population if type == "population").

Here, the underlying variables are "cases" and "population", but they are represented by the pair (type, count) rather than separate columns.

```
table2
```

```
# A tibble: 12 x 4
   country      year type            count
   <chr>       <dbl> <chr>           <dbl>
 1 Afghanistan  1999 cases             745
 2 Afghanistan  1999 population   19987071
 3 Afghanistan  2000 cases            2666
 4 Afghanistan  2000 population   20595360
 5 Brazil       1999 cases           37737
 6 Brazil       1999 population  172006362
 7 Brazil       2000 cases           80488
 8 Brazil       2000 population  174504898
 9 China        1999 cases          212258
10 China        1999 population 1272915272
```

```
11 China        2000 cases          213766
12 China        2000 population 1280428583
```

In table3, each row is again one country–year observation, but the disease information is expressed as a rate instead of separate counts.

Columns:

- country: Country name, same as above.
- year: Calendar year, same as above.
- rate: Disease rate, defined conceptually as cases divided by population for that country and year (often written in a "cases / population" style in the example data).

```
table3
```

```
# A tibble: 6 x 3
  country      year rate
  <chr>       <dbl> <chr>
1 Afghanistan  1999 745/19987071
2 Afghanistan  2000 2666/20595360
3 Brazil       1999 37737/172006362
4 Brazil       2000 80488/174504898
5 China        1999 212258/1272915272
6 China        2000 213766/1280428583
```

So, across table1, table2, and table3, country and year always define the observation; what changes is whether the absolute counts are stored in separate columns, as long format type–value pairs, or as a single rate variable.

**2. Sketch out the process you'd use to calculate the rate for table2 and table3. You will need to perform four operations:**

- Extract the number of TB cases per country per year.
- Extract the matching population per country per year.
- Divide cases by population, and multiply by 10000.
- Store back in the appropriate place.

**table2**

table2 has columns country, year, type (either "cases" or "population"), and count. Conceptually you want to "widen" it so you get separate cases and population columns, then compute the rate.

1. Extract TB cases per country–year

    - Look at all rows where type is “cases”.
    - For each country and year combination, take the count value from these rows as the number of TB cases.
    - Mentally, you can imagine building a temporary list or table: (country, year, cases).

2. Extract matching population per country–year

- Now look at all rows where type is “population”.
- For each country and year, take the count value as the total population.
- Conceptually you have a second list: (country, year, population).
- Match these two lists together by country and year so that, for every country–year, you have both cases and population side by side.

3. Compute rate = cases / population * 10,000

- For each matched country–year record, divide the number of cases by the population to get a proportion.
- Multiply that proportion by 10,000 to convert it to “cases per 10,000 people.”
- Store that numeric result as a new value called rate for that country–year.

4. Store back in the appropriate place

- Decide on the structure you want going forward. Typically, you’d create a table with columns country, year, cases, population, and rate.
- Conceptually, this is “table2 reshaped”: one row per country–year, the new rate column added there.
- If you need to keep the original long format for some reason, you’d treat this as a derived table, but the core idea is that rate lives alongside the corresponding cases and population for each country–year.

**table3**

table3 has columns country, year, and rate, but here rate is stored as a textual composite like “cases/population” rather than a numeric rate per 10,000.

1. Extract TB cases per country–year

- Start from the composite rate value in each row (e.g. “745/19987071”).

- Conceptually split that value into two pieces separated by "/": the first part is the number of cases, the second part is the population.
- Treat the first piece as the TB cases count for that row's country–year, forming (country, year, cases).

2. Extract matching population per country–year

- For the same row, take the second piece of the split string as the population.
- Convert it to a numeric population value and pair it with the cases for that country–year, i.e. (country, year, cases, population).

3. Compute rate = cases / population * 10,000

- For each row, divide the numeric cases by the numeric population to get a proportion.
- Multiply that proportion by 10,000 to express it as "cases per 10,000 people."
- This gives you a proper numeric rate variable derived from the decomposed cases and population.

4. Store back in the appropriate place

- Replace or augment the original rate column with your newly computed numeric rate per 10,000.
- Keep cases and population as their own columns so the table now looks like: country, year, cases, population, rate.
- At this point, table3 is structurally similar to the reshaped version of table2, and you can use it in the same way for analysis or visualization.

### 5.3 Lengthening data

#### Data in column names

The billboard dataset records the billboard rank of songs in the year 2000:

```
billboard
```

```
# A tibble: 317 x 79
   artist     track date.entered   wk1   wk2   wk3   wk4   wk5   wk6   wk7   wk8
   <chr>      <chr> <date>       <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
 1 2 Pac      Baby~ 2000-02-26      87    82    72    77    87    94    99    NA
 2 2Ge+her    The ~ 2000-09-02      91    87    92    NA    NA    NA    NA    NA
 3 3 Doors D~ Kryp~ 2000-04-08      81    70    68    67    66    57    54    53
```

```
 4 3 Doors D~ Loser 2000-10-21      76    76    72    69    67    65    55    59
 5 504 Boyz   Wobb~ 2000-04-15      57    34    25    17    17    31    36    49
 6 98^0       Give~ 2000-08-19      51    39    34    26    26    19     2     2
 7 A*Teens    Danc~ 2000-07-08      97    97    96    95   100    NA    NA    NA
 8 Aaliyah    I Do~ 2000-01-29      84    62    51    41    38    35    35    38
 9 Aaliyah    Try ~ 2000-03-18      59    53    38    28    21    18    16    14
10 Adams, Yo~ Open~ 2000-08-26      76    76    74    69    68    67    61    58
# i 307 more rows
# i 68 more variables: wk9 <dbl>, wk10 <dbl>, wk11 <dbl>, wk12 <dbl>,
#   wk13 <dbl>, wk14 <dbl>, wk15 <dbl>, wk16 <dbl>, wk17 <dbl>, wk18 <dbl>,
#   wk19 <dbl>, wk20 <dbl>, wk21 <dbl>, wk22 <dbl>, wk23 <dbl>, wk24 <dbl>,
#   wk25 <dbl>, wk26 <dbl>, wk27 <dbl>, wk28 <dbl>, wk29 <dbl>, wk30 <dbl>,
#   wk31 <dbl>, wk32 <dbl>, wk33 <dbl>, wk34 <dbl>, wk35 <dbl>, wk36 <dbl>,
#   wk37 <dbl>, wk38 <dbl>, wk39 <dbl>, wk40 <dbl>, wk41 <dbl>, wk42 <dbl>, ...
```

In this dataset, each observation is a song. The first three columns (artist, track and date.entered) are variables that describe the song. Then we have 76 columns (wk1-wk76) that describe the rank of the song in each week1. Here, the column names are one variable (the week) and the cell values are another (the rank).

To tidy this data, we'll use pivot_longer():

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank"
  )
```

```
# A tibble: 24,092 x 5
   artist track                   date.entered week   rank
   <chr>  <chr>                   <date>       <chr> <dbl>
 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
 7 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk7      99
 8 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk8      NA
 9 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk9      NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk10     NA
# i 24,082 more rows
```